LAB SHEET – 05
MOHD RAZA
CU24250044

Index.js

```
require('dotenv').config();  const
express = require('express'); const app =
express(); const PORT =
process.env.PORT || 4000; // 11. Custom
Global Logger Middleware const
requestLogger = (req, res, next) => {
const mestamp = new Date().toISOString();
  console.log(`[${ mestamp}] ${req.method} ${req.url}`);
next();
};
app.use(requestLogger);
// 14. Enable JSON and URL-encoded body parsers
app.use(express.json());
app.use(express.urlencoded({  extended:  true  }));  //
Part I: Serve Sta c Files (public folder)
app.use(express.static('public'));
//  Part  F:  Import  Modular  Routers  const
studentsRouter=require('./routes/students'); const
booksRouter            =
require('./routes/books');  // Part F:Mount Routers
under /api prefixes
app.use('/api/students',studentsRouter);
app.use('/api/books', booksRouter);
// Part A & B: Basic Server & Rou ng
// GET / (Part A) app.get('/', (req,
res) => {     res.send('Express Lab
Running');
});

// 5. GET /about app.get('/about',
(req, res) => {
res.json({     name: 'Your
Name',      rollNumber:
'123456'
  });    });
// 6. GET /courses app.get('/courses',
(req, res) => {     res.json(['Full Stack Web Development', 'Data Structures', 'Database
Management']);
});
// 7. POST /echo
app.post('/echo', (req, res) => {
res.json(req.body); });  // 9.
GET /search app.get('/search',
(req, res) => {   const { name,
age } = req.query;
  res.json({ name, age });
});
// 10. GET /products/:category/:id
app.get('/products/:category/:id', (req, res) => {   const
{ category, id } = req.params;
  res.json({ category, id });
```

```
// 12. API Key Checker Middleware for /admin
routes  const apiKeyAuth = (req, res, next) => {
const apiKey = req.headers['x-api-key'];   if (!apiKey
|| apiKey !== 'my-secret-key') {
return res.status(403).json({ error: '403 Forbidden: Missing or invalid x-api-key' });  }   next();
};
// Apply apiKeyAuth only to routes star ng with /admin
app.use('/admin', apiKeyAuth);

// 13. Protected Route  app.get('/admin/dashboard',
(req, res) => {
  res.json({ message: 'Welcome to the admin dashboard!' });
});
//   15.   POST   /register   (JSON   body)
app.post('/register', (req, res) => {   const {
name, email, password } = req.body;
  res.json({ message: `Successfully registered user: ${name}`, email });
});
// 16. POST /contact (URL-encoded form submission)
app.post('/contact', (req, res) => {
console.log('Form Submission:', req.body);
res.send('Thank you for contac ng us!');  });  //
22. 404 Unmatched Route Handler
app.use((req, res, next) => {
  res.status(404).json({ error: "Route not found" });});
// 23. Centralized Error-Handling Middleware
app.use((err, req, res, next) => {
console.error(err.stack); const statusCode = err.status
|| 500;   res.status(statusCode).json({ error:
err.message || "Internal Server Error" });});
app.listen(PORT, () => {
  console.log(`Server is running on h p://localhost:${PORT}`);
});
```

Books.js

```
//routes/books.js  const
express = require('express');
const router = express.Router();

let books = [
  { id: 1, tle: 'Node.js Guide' },
  { id: 2, tle: 'Express Fundamentals' }
];

// GET /api/books (All books)
router.get('/', (req, res) => {
  res.json(books);
});

// GET /api/books/:id (Single book)
router.get('/:id', (req, res) => {
  const book = books.find(b => b.id === parseInt(req.params.id));
if (!book) return res.status(404).json({ error: 'Book not found' });
res.json(book);
});

// POST /api/books (Add a
book) router.post('/',(req,res)=>{
const newBook = {       id:
books.length + 1,
```

```
   tle: req.body. tle
};
books.push(newBook)
;
res.status(201).json(newBook); });

module.exports = router;
```

task.js

```
const express = require('express');  const
router = express.Router();

let tasks = [
  { id: 1, tle: 'Learn Express', completed: false },
  { id: 2, tle: 'Build REST API', completed: true }
];

// GET all  router.get('/',
(req, res) => {
res.json(tasks);
});

// GET single with explicit 404 error passing via next(err)
router.get('/:id', (req, res, next) => {    const task = tasks.find(t
=> t.id === parseInt(req.params.id));
if (!task) {
   const error = new Error('Task not
found');      error.status = 404;      return
next(error);
  }
res.json(task);
});

// POST create  router.post('/',
(req, res, next) => {
if (!req.body. tle) {      const error = new
Error('Title isrequired');     error.status =
400;
   return next(error);
  }
  const newTask = {      id: tasks.length ?
tasks[tasks.length - 1].id + 1 : 1,      tle: req.body.
title,      completed: req.body.completed || false
  };
  tasks.push(newTask);
res.status(201).json(newTask);
});

// PUT update
router.put('/:id', (req, res, next) => {    const task = tasks.find(t
=> t.id === parseInt(req.params.id));
if (!task) {
   const error = new Error('Task not
found');      error.status = 404;      return
next(error);
  }    task. title = req.body. tle || task. title;    task.completed = req.body.completed !==
undefined ? req.body.completed : task.completed;
res.json(task);  });
```

```
// DELETE remove
router.delete('/:id', (req, res, next) => {    const index =
tasks.findIndex(t => t.id === parseInt(req.params.id));
if (index === -1) {
   const error = new Error('Task not
found');      error.status = 404;      return
next(error);
  }
 const deletedTask = tasks.splice(index, 1);
 res.json({ message: 'Task deleted successfully', deletedTask });
});

module.exports = router;
```







